AI USAGE · COMPANION TO THE DESIGN DOCUMENT

Where I Overrode the AI

A record of the decisions I took back from the model on this exercise — what it got wrong, how I caught it, and what changed as a result.

I used Claude throughout this exercise, mostly as a way to generate options quickly and pressure-test my own reasoning. The useful record isn't that I used it — it's the handful of places where its output was wrong, incomplete, or stale, and how those were caught. Those are below, in the order they happened.

01 Where it helped

Three things, honestly:

- **Enumerating options fast.** For each of the three problems, I asked for the realistic approaches with tradeoffs rather than a recommendation. Choosing among four framed options is quicker than generating them, and it keeps the decision mine.
- **Spec and licensing lookup.** RFC 8693 parameter names, RFC 8707's `invalid_target`, RFC 6749 error semantics, Duende's current licensing tiers. All verified against primary sources before they influenced anything.
- **Test scaffolding.** Once the decision function was pure, generating the table of cases was mechanical. The *design* that made it mechanical — no I/O in the policy — was the part worth thinking about.

02 Where I corrected it

Redis throughput — a recommendation with no arithmetic behind it

I ASKED

> "So you think that with Redis is not enough to support that concurrency of authorization checks?"

WHAT CHANGED

It had recommended an in-process L1 cache in front of Redis. When I pushed for the numbers, the recommendation didn't survive them: a single Redis instance handles roughly 100k+ simple reads per second against a requirement of maybe 30–50k, spread across many distinct keys. **L1 came out of the design** as a blanket recommendation and stayed only as a documented extension with named triggers — hot keys, cross-AZ latency, dependency coupling. One selective use survived: caching just the firm policy version, which is low-cardinality and read on every check.

Permissions in tokens — rejected before it was argued for

> I SAID
>
> "I don't think it is a good idea to put these PDP decisions into tokens."

WHAT CHANGED

This was on the table as one of three placements for the decision point. Ruling it out produced **the governing rule the rest of the design hangs on**: if a fact can change between the moment a token is minted and the moment it is used, it does not belong in the token. That rule later decided something else on its own — RFC 8693 §4.4 defines a `may_act` claim for recording who may act for a subject, and we don't use it, because an embedded claim freezes the decision at issue time. The entitlement lookup runs at exchange time instead.

Priority drift — code before the primary deliverable

> I INTERRUPTED WITH
>
> "But before that we need to work on the design document."

WHAT CHANGED

It had started scaffolding the .NET solution. The brief marks Part 1 as the primary focus and Part 2 as optional, so **the design document was written first** and the implementation resumed afterward. Worth noting as a pattern: the model optimises for the task in front of it, not for the relative weight of the deliverables.

Stale documentation it never volunteered

> I ASKED
>
> "Did you implement changes to DESIGN.md during or after the implementation?"

WHAT CHANGED

It had not. File timestamps showed the design document written at 01:48 and every source file after 17:52. In between, the implementation had acquired **a fifth confused-deputy control the document never mentioned** (a delegation depth cap), plus a deliberate deviation from RFC 8693 — `audience` is optional in the spec and required here — that existed only in a code comment. The brief grades "where you'd deviate from spec, and why," so that omission would have cost something. The design document now carries all five controls and both deviations.

03 Where AI should not be trusted here

Four failure modes showed up in this session specifically. All four share a shape: the output *reads* correct.

> THE ONE THAT WOULD HAVE COST THE MOST
>
> The test suite went green at 27 passing — while the most important test in it was passing for the wrong reason. `Refuses_to_mint_for_a_user_outside_the_clients_own_firm` was returning the right error code, but from the *consent store* (that user simply had no grant record), never reaching the firm-boundary invariant it exists to protect. Reading the log output caught it; the pass count never would have. The fix was to seed the consent store *deliberately wrong* — granting Acme's client consent for a Globex user — so the structural check is what refuses. That seed is now load-bearing, and commented as such.

Library capability claims	It stated a difference in RFC 8693 support between Duende and OpenIddict that does not exist — neither ships it; both need a custom grant handler. That claim was shaping a library decision before it was checked.
Capacity estimates	Architectural reflexes arrive with the same confidence as arithmetic. Ask for the numbers; the recommendation may not survive them.
Green test suites	Authorization tests can pass for the wrong reason. A passing assertion proves an outcome, not a mechanism.
Its own stale output	It will not tell you that something it wrote three hours ago no longer matches what it built since. Nothing prompts a re-audit except asking for one.

Underneath all of it is one property of this domain: **authorization logic fails permissively and silently.** A scope-narrowing bug does not throw — it grants. There is no stack trace, no failing request, nothing that surfaces during ordinary use. That makes generated code here categorically riskier than generated code in a domain where mistakes crash.

04 How I'd guide a team using AI on this system

- **Generate options, not decisions.** Ask for the realistic approaches with tradeoffs and choose yourself. The model is good at recall and enumeration and confident about things it has not verified.
- **Design for testability, then let it write the tests.** Keeping the decision function pure — no I/O, no clock — is what makes the security-critical path exhaustively table-testable. That's a human design choice, and it's what makes accepting generated tests safe.
- **Write the negative tests first, then read why they pass.** In authorization, the refusals carry the weight, and a green suite is not evidence until you have seen the mechanism that produced it.
- **Verify every claim about a library or a spec.** Especially the ones that decide something. Two of the four corrections above were unverified assertions stated fluently.
- **Re-audit earlier artifacts after implementing.** Documents drift from code silently, and the model has no sense that its own earlier output has gone stale.

Companion to the design document and the token exchange implementation. The complete session transcript is available on request; this page is the curated version, covering the exchanges where the outcome changed.